task6.java

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;
public class task6 {
    private static final int THREADS = 8;  9 usages
    private static final int INCREMENTS_PER_THREAD = 1_000_000;  5 usages
    private static final long TOTAL_OPS =  5 usages
            (long) THREADS * INCREMENTS_PER_THREAD;
    private static class Result {  6 usages
        long counter;  5 usages
        double timeMs;  3 usages
        double throughput;  5 usages
        Result(long counter, double timeMs) {  2 usages
            this.counter = counter;
            this.timeMs = timeMs;
            this.throughput = TOTAL_OPS / (timeMs / 1000.0);
        }
    }
    private static Result benchmarkMutex() throws InterruptedException {  1 usage
        ReentrantLock lock = new ReentrantLock();
        long[] counter = {0};

        CountDownLatch ready = new CountDownLatch(THREADS);
```

task6.java

```
9   public class task6 {
23      }
24  @   private static Result benchmarkMutex() throws InterruptedException {  1 us
25          ReentrantLock lock = new ReentrantLock();
26          long[] counter = {0};
27          CountDownLatch ready = new CountDownLatch(THREADS);
28          CountDownLatch start = new CountDownLatch(1);
29          CountDownLatch done = new CountDownLatch(THREADS);
30          List<Thread> workers = new ArrayList<>();
31          for (int i = 0; i < THREADS; i++) {
32              Thread worker = new Thread(() -> {
33                  ready.countDown();
34                  try {
35                      start.await();
36                      for (int j = 0; j < INCREMENTS_PER_THREAD; j++) {
37                          lock.lock();
38                          try {
39                              counter[0]++;
40                          } finally {
41                              lock.unlock();
42                          }
43                      }
44                  } catch (InterruptedException e) {
45                      Thread.currentThread().interrupt();
46                  } finally {
47                      done.countDown();
48                  }
49              });
50
51              workers.add(worker);
52              worker.start();
```

```
9   public class task6 {
24      private static Result benchmarkMutex() throws InterruptedException {  1 us
51          workers.add(worker);
52          worker.start();
53      }
54
55      ready.await();
56      long begin = System.nanoTime();
57      start.countDown();
58      done.await();
59      long end = System.nanoTime();
60
61      for (Thread worker : workers) {
62          worker.join();
63      }
64
65      double timeMs = (end - begin) / 1_000_000.0;
66      return new Result(counter[0], timeMs);
67  }
68
69 @    private static Result benchmarkAtomic() throws InterruptedException {  1
70      AtomicInteger counter = new AtomicInteger( initialValue: 0);
71
72      CountDownLatch ready = new CountDownLatch(THREADS);
73      CountDownLatch start = new CountDownLatch(1);
74      CountDownLatch done = new CountDownLatch(THREADS);
75
76      List<Thread> workers = new ArrayList<>();
77
78      for (int i = 0; i < THREADS; i++) {
79          Thread worker = new Thread(() -> {
```

```
 9  public class task6 {
69      private static Result benchmarkAtomic() throws InterruptedException {
78          for (int i = 0; i < THREADS; i++) {
79              Thread worker = new Thread(() -> {
80                  ready.countDown();
81
82                  try {
83                      start.await();
84
85                      for (int j = 0; j < INCREMENTS_PER_THREAD; j++) {
86                          counter.incrementAndGet();
87                      }
88                  } catch (InterruptedException e) {
89                      Thread.currentThread().interrupt();
90                  } finally {
91                      done.countDown();
92                  }
93              });
94
95              workers.add(worker);
96              worker.start();
97          }
98
99          ready.await();
100         long begin = System.nanoTime();
101         start.countDown();
102         done.await();
103         long end = System.nanoTime();
104
105         for (Thread worker : workers) {
106             worker.join();
```

```
 9    public class task6 {
69        private static Result benchmarkAtomic() throws InterruptedException {  1 usa
105           for (Thread worker : workers) {
106               worker.join();
107           }
108
109           double timeMs = (end - begin) / 1_000_000.0;
110           return new Result(counter.get(), timeMs);
111       }
112
113       public static void main(String[] args) {
114           try {
115               System.out.println("TASK 6 — Mutex vs. Atomic CAS");
116               System.out.println("Threads: " + THREADS);
117               System.out.println(
118                       "Increments per thread: " + INCREMENTS_PER_THREAD);
119               System.out.println("Total operations per test: " + TOTAL_OPS);
120               System.out.println();
121               Result mutex = benchmarkMutex();
122               Result atomic = benchmarkAtomic();
123               if (mutex.counter != TOTAL_OPS ||
124                       atomic.counter != TOTAL_OPS) {
125                   System.err.println("ERROR: Counter verification failed.");
126                   System.exit( status: 1);
127               }
128
129               double ratio = atomic.throughput / mutex.throughput;
130
131               List<String> lines = List.of(
132                       ...elements: "TASK 6 — Mutex vs. Atomic CAS",
133                       "Threads: " + THREADS,
```

```
 9  public class task6 {
113      public static void main(String[] args) {
129          double ratio = atomic.throughput / mutex.throughput;
130
131          List<String> lines = List.of(
132                  ...elements: "TASK 6 — Mutex vs. Atomic CAS",
133                  "Threads: " + THREADS,
134                  "Increments per thread: " + INCREMENTS_PER_THREAD,
135                  "Total operations per test: " + TOTAL_OPS,
136                  "",
137                  String.format(
138                          "Mutex Execution Time: %.2f ms", mutex.timeMs),
139                  String.format(
140                          "Mutex Throughput: %.2f ops/sec",
141                          mutex.throughput),
142                  "Mutex Final Counter: " + mutex.counter,
143                  "",
144                  String.format(
145                          "Atomic CAS Execution Time: %.2f ms",
146                          atomic.timeMs),
147                  String.format(
148                          "Atomic CAS Throughput: %.2f ops/sec",
149                          atomic.throughput),
150                  "Atomic Final Counter: " + atomic.counter,
151                  "",
152                  String.format("Throughput Ratio: %.4fx", ratio)
153          );
154
155          String output = String.join( delimiter: "\n", lines);
156          System.out.println(output);
```

```
                "",
                String.format("Throughput Ratio: %.4fx", ratio)
            );

            String output = String.join( delimiter: "\n", lines);
            System.out.println(output);

            Files.writeString(
                Path.of( first: "task6_output.txt"),
                 csq: output + System.lineSeparator()
            );

            System.out.println("\nResults saved to task6_output.txt");

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.err.println("Benchmark interrupted.");
            System.exit( status: 1);
        } catch (IOException e) {
            System.err.println("Could not save output: " + e.getMessage());
            System.exit( status: 1);
        }
    }
}

```